Incoming URL	Maps to
https://domain.com/city/london	c:\website\city\london.cshtml
https://domain.com/booking/checkout	c:\website\booking\checkout.cshtml
https://domain.com/login	c:\website\login.cshtml

This chapter will explore how Razor Pages uses *page-based routing*—mapping between URLs and file paths as the basis for routing within an application. It will also explore how you can use configuration to customize how URLs map to endpoints when a direct relationship between the URL and the location on disk is insufficient for your needs.

The one-to-one correspondence between the URL and the file path is easy to reason about, but it is very limiting on its own. The first example in table 4.1 hints at a page that provides information about London as a vacation destination. If you also wanted to provide information about Paris (https://domain.com/city/paris), you would have to add another page to the city folder named paris.cshtml. As discussed in previous chapters, this is not a sustainable way to build an application. In the last chapter, we looked at passing data in the URL as a query string value. This chapter will explore passing data as part of the URL itself and getting a single page to respond differently based on that data.

URLs are considered part of your web application's UI. So you need to be able to exercise as much control over them as you have over the generation of HTML within your application. The routing system controls how the framework generates URLs to endpoints, but there are customization options available. You will see how to use them to ensure your URLs are descriptive and human readable.

In the last chapter, we looked at some examples of using Razor syntax to iterate over a list of cities. To demonstrate the concepts in this chapter and in